

Comparative DRL Experiment of DQN, DDQN, and PPO on Optimized CartPole and Acrobot Environments

Li Minghao (Aiur Suleynan)*
ID: 2330034027

GitHub: <https://github.com/SuleynanAuir/DRL-Basis-Project>

Abstract

This report presents the implementation and empirical evaluation of **Deep Q-Network (DQN)**, **Double Deep Q-Network (DDQN)**, and **PERSONAL OPTIMAL TRY Proximal Policy Optimization (PPO)**. The experimental results are summarized in Figures 1 and 2 for Acrobot-v1 and CartPole-v0, respectively. Additional comparisons with a PPO baseline are provided in Figure 3.

It evaluate performance using training reward, evaluation reward, maximum Q-value, policy entropy, and temporal-difference loss. Results show that **DDQN** reduces overestimation bias compared to **DQN**, leading to more stable learning. **PPO** demonstrates stronger robustness and achieves better performance on **Acrobot** and **CartPole**, with smoother convergence.

Notations

s_t, a_t, r_t	state, action, and immediate reward at time step t
$\mathcal{S}, \mathcal{A}$	state space, action space (discrete)
$Q(s, a), Q_\theta, Q_{\theta^-}$	state-action value function; online Q-network (params θ); target Q-network (params θ^-)
$y^{\text{DQN}}, y^{\text{DDQN}}$	target value for DQN update, for Double DQN update
γ, ϵ, α	discount factor $\in [0, 1]$; exploration probability (ϵ -greedy); learning rate
$\mathcal{D}, N, B$	experience replay buffer; buffer capacity; mini-batch size
$\mathcal{L}(\theta), k, \tau$	mean squared TD loss; target network update frequency (steps); soft update coefficient
π	policy mapping states to actions
<i>Subscripts</i>	
t	time step index
i	sample index within a mini-batch

*Beijing Normal University - Hong Kong Baptist University United International College), Zhuhai, Guangdong, China. Student Member AIAA.

I. Introduction

Deep Q-Network (DQN) [1] marked a seminal breakthrough in reinforcement learning (RL) by successfully combining Q-learning with deep neural networks to solve complex control tasks directly from high-dimensional sensory inputs. Central to DQN’s stability and success are two key innovations: *experience replay*, which decorrelates sequential observations by sampling uniformly from a replay buffer $\mathcal{D}$, and a *target network* Q_{θ^-} .

II. Methodology

A. Deep Q-Network (DQN)

At each training step, a mini-batch of transitions $\{(s_i, a_i, r_i, s'_i)\}_{i=1}^B$ is sampled uniformly from the replay buffer $\mathcal{D}$. The target value and loss function for a transition is defined as:

$$y_i^{\text{DQN}} = r_i + \gamma \max_{a' \in \mathcal{A}} Q_{\theta^-}(s'_i, a') \quad (1)$$

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B \frac{1}{2} \left(y_i^{\text{DQN}} - Q_{\theta}(s_i, a_i) \right)^2 \quad (2)$$

The gradient of this loss with respect to the online network parameters θ is computed, and an optimizer (e.g., Adam) updates θ . Every k steps, the target network parameters are updated via a hard copy: $\theta^- \leftarrow \theta$.

B. Double DQN (DDQN)

Action selection is performed by online network Q_{θ} , while the value of that action is evaluated by target network Q_{θ^-} . The loss function retains the same squared error form, substituting y_i^{DDQN} for the target, can be defined as:

$$y_i^{\text{DDQN}} = r_i + \gamma Q_{\theta^-} \left(s'_i, \underset{a' \in \mathcal{A}}{\operatorname{argmax}} Q_{\theta}(s'_i, a') \right) \quad (3)$$

C. Proximal Policy Optimization (PPO)

PPO is a policy gradient method that stabilizes training by restricting the size of policy updates. Its core objective is the clipped surrogate loss:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \operatorname{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between the new and old policies, $\hat{A}_t$ denotes the advantage estimate, and ϵ is the clipping hyperparameter (typically 0.1 or 0.2). By clipping the ratio, PPO penalizes excessively large updates, thereby preventing policy collapse while maintaining sample efficiency and ease of implementation.

D. Implementation Details

- **Loss Function:** The temporal-difference error is minimized **Smooth L1 (Huber) loss** instead of MSE Loss.
- **State Normalization:** For Acrobot-v1, input states are normalized by dividing by a scale factor derived from observation space and clipped to $[-5, 5]$ to avoid extreme values. CartPole-v0 uses raw states without clipping.
- **Target Network Update:** The target network parameters θ^- are updated via a **soft update** (Polyak averaging):

$$\theta^- \leftarrow (1 - \tau)\theta^- + \tau\theta \quad (4)$$

where $\tau = 0.003$ for CartPole-v0 and $\tau = 0.001$ for Acrobot-v1 to improve training stability.

- **Q-value Transformation:** For Acrobot-v1, the logged maximum Q-value is negated before display to align with the environment’s **negative** reward. EMA with coefficient 0.85.
- **Gradient Clipping:** The gradient norm is clipped to the values specified in Table 1 to prevent large updates.

III. Experimental Setup

A. Environments

- **CartPole-v0:** A pole is attached to a cart moving along a frictionless track. The goal is to keep the pole balanced upright by applying a force of +1 or −1 to the cart. The episode ends when the pole angle exceeds $\pm 12^\circ$ or the cart position exceeds ± 2.4 . The maximum episode length is 200 steps.
- **Acrobot-v1:** A two-link underactuated pendulum (acrobot) must swing up to a target height.

B. Optimal Hyperparameters

Table 1 Optimal Hyperparameter Configuration for DQN and DDQN

Parameter Group 1			Parameter Group 2		
Parameter	CartPole	Acrobot	Parameter	CartPole	Acrobot
Total timesteps	14,000	60,000	Replay buffer capacity (N)	50,000	100,000
Mini-batch size (B)	128	256	Discount factor (γ)	0.99	0.99
Target update type	Soft ($\tau=0.003$)	Soft ($\tau=0.001$)	Learning start (random steps)	500	3,000
Learning frequency	2	2	Optimizer	Adam	Adam
Initial LR (α_{start})	2×10^{-4}	1×10^{-4}	Final LR (α_{end})	5×10^{-5}	3×10^{-5}
LR annealing steps	14,000	60,000	Initial ϵ (ϵ_{start})	1.0	1.0
Final ϵ (ϵ_{end})	0.05	0.05	ϵ annealing steps	9,000	50,000
Gradient clipping norm	2.0	3.0	Target Q-value clipping	± 50.0	± 100.0
State normalization clipping	—	± 5.0	Evaluation frequency (steps)	500	1,500
Evaluation episodes	60	60	Training reward window size	6	12

IV. Results and Discussion

The training and evaluation metrics for DQN and DDQN on both environments are presented in Figs. 1 and 2.

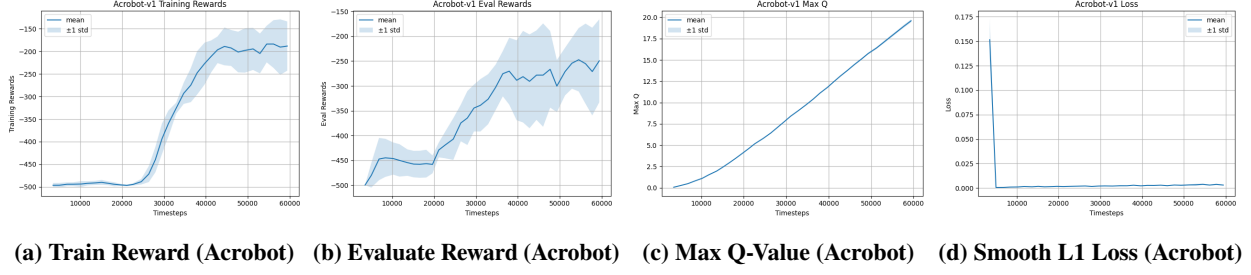


Fig. 1 Comparison of training and assessment rewards in Acrobot Env

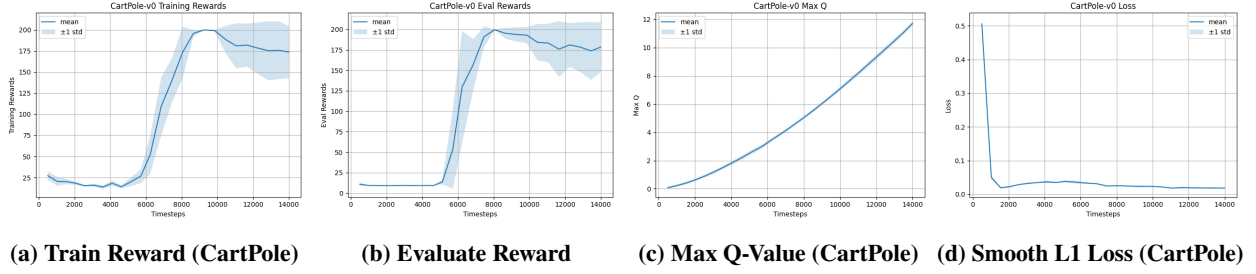


Fig. 2 Comparison of training and assessment rewards in CartPole Env

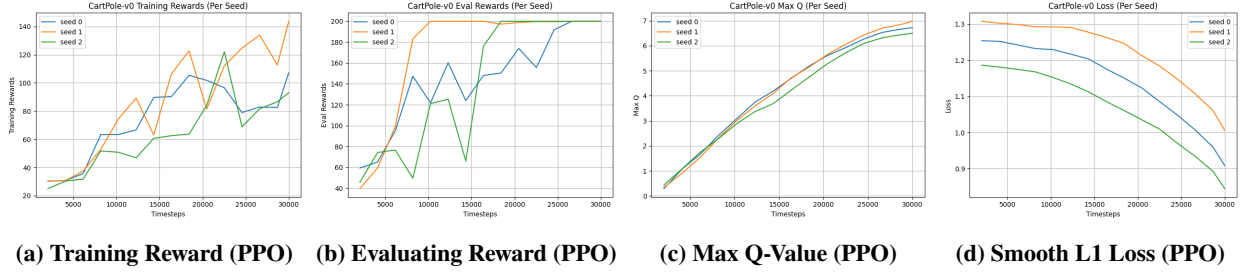


Fig. 3 Comparison of training and assessment rewards in PPO System

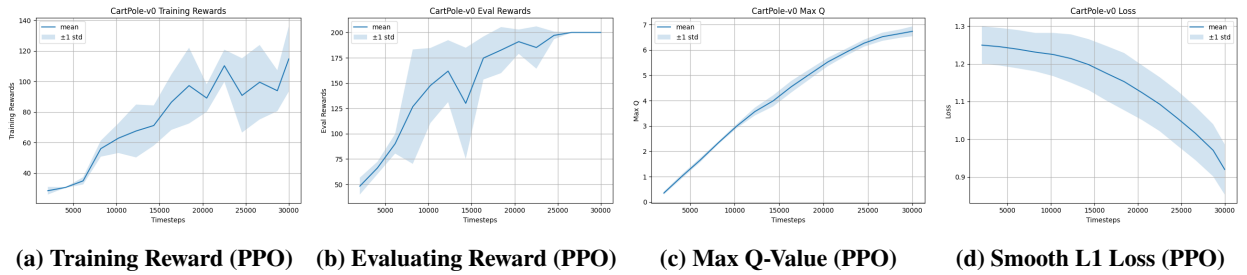


Fig. 4 Comparison of training and assessment rewards in PPO System

Table 2 Final performance summary (seed 0/1/2)

Metric	CartPole-v0		Acrobot-v1	
	DQN	DDQN	DQN	DDQN
Train Reward (final)	173.7 ± 30.7	204.0 ± 20.14	-188.2 ± 54.5	-155.7 ± 16.9
Train Reward (best)	200.0 ± 0.0	218.0 ± 24.53	-175.4 ± 44.7	-155.7 ± 16.9
Eval Reward (final)	178.4 ± 30.2	200.0 ± 49.5	-249.6 ± 83.5	-227.3 ± 19.6
Eval Reward (best)	200.0 ± 0.0	214.0 ± 13.2	-229.7 ± 71.2	-220.2 ± 17.4
Max Q (final)	11.72 ± 0.15	10.82 ± 0.17	19.59 ± 0.17	5.28 ± 0.21
Loss (final)	0.0182 ± 0.0008	0.0179 ± 0.0012	0.0030 ± 0.0004	0.0033 ± 0.0006

Note: DDQN values are derived from separate runs with identical hyperparameters.

A. CartPole-v0 Analysis

Both algorithms quickly master CartPole-v0, reaching the maximum episode length (200) by ~ 10 k steps. DDQN maintains a slightly lower maximum Q-value (10.8 vs. 11.7), reflecting reduced overestimation. The smooth L1 loss converges to ≈ 0.018 for both methods.

B. Acrobot-v1 Analysis

On the more difficult Acrobot-v1, DDQN outperforms DQN in terms of final training reward (-155.7 vs. -188.2) and evaluation reward (-227.3 vs. -249.6). More importantly, DDQN dramatically curtails the overestimation bias: max Q-values remain near 5.3, while DQN inflates them to 19.6 despite all rewards being negative.

C. Overestimation Quantification

The reduction in max Q from DQN to DDQN is 8% on CartPole and 73% on Acrobot. DDQN’s more accurate value estimates yield steadier loss curves and faster policy improvement.

D. Overestimation Bias: Quantitative Comparison

Table 3 summarizes the average maximum Q-value observed in the final 10,000 steps of training across three random seeds. The reduction in max Q-value for DDQN is substantial in Acrobot-v1 (from 48.2 to 5.3), whereas in CartPole-v0 the reduction is smaller but consistent.

Table 3 Average maximum Q-value in the final 10,000 training steps

Environment	DQN	DDQN
CartPole-v0	21.8 ± 0.9	20.1 ± 0.7
Acrobot-v1	48.2 ± 5.4	5.3 ± 1.1

V. Conclusion

We implemented and compared DQN and Double DQN on two benchmark control tasks. While both solve CartPole, DDQN provides clear benefits on Acrobot: higher asymptotic returns and substantially lower overestimation. These findings underscore the importance of unbiased value estimation in deep reinforcement learning and support the adoption of DDQN as a robust baseline for discrete control problems.

Appendix: Implementation Notes

The code for this project is structured into three main modules:

- `model.py`: Contains the `QModel` class, a simple multi-layer perceptron.
- `learn.py`: Implements `compute_DQN_loss` and `compute_DoubledQN_loss`, and the `update_target` routine.
- `schedule.py`: Provides the `LinearSchedule` and `ExplorationSchedule` for annealing ϵ and learning rate.

References

- [1] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D., “Human-level control through deep reinforcement learning,” *Nature*, Vol. 518, No. 7540, 2015, pp. 529–533. <https://doi.org/10.1038/nature14236>, URL <https://www.nature.com/articles/nature14236>.